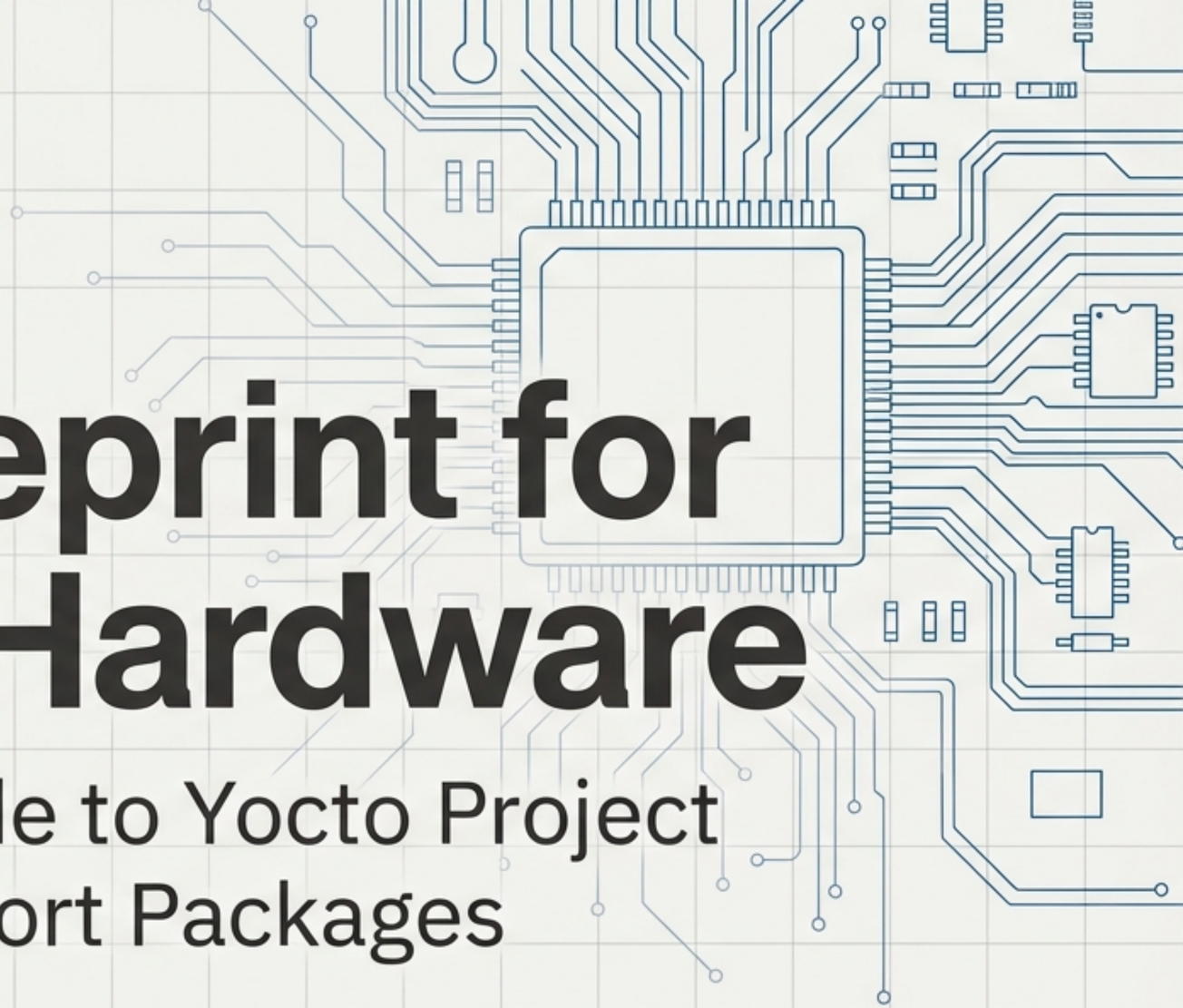


The Blueprint for Custom Hardware

A Designer’s Guide to Yocto Project Board Support Packages

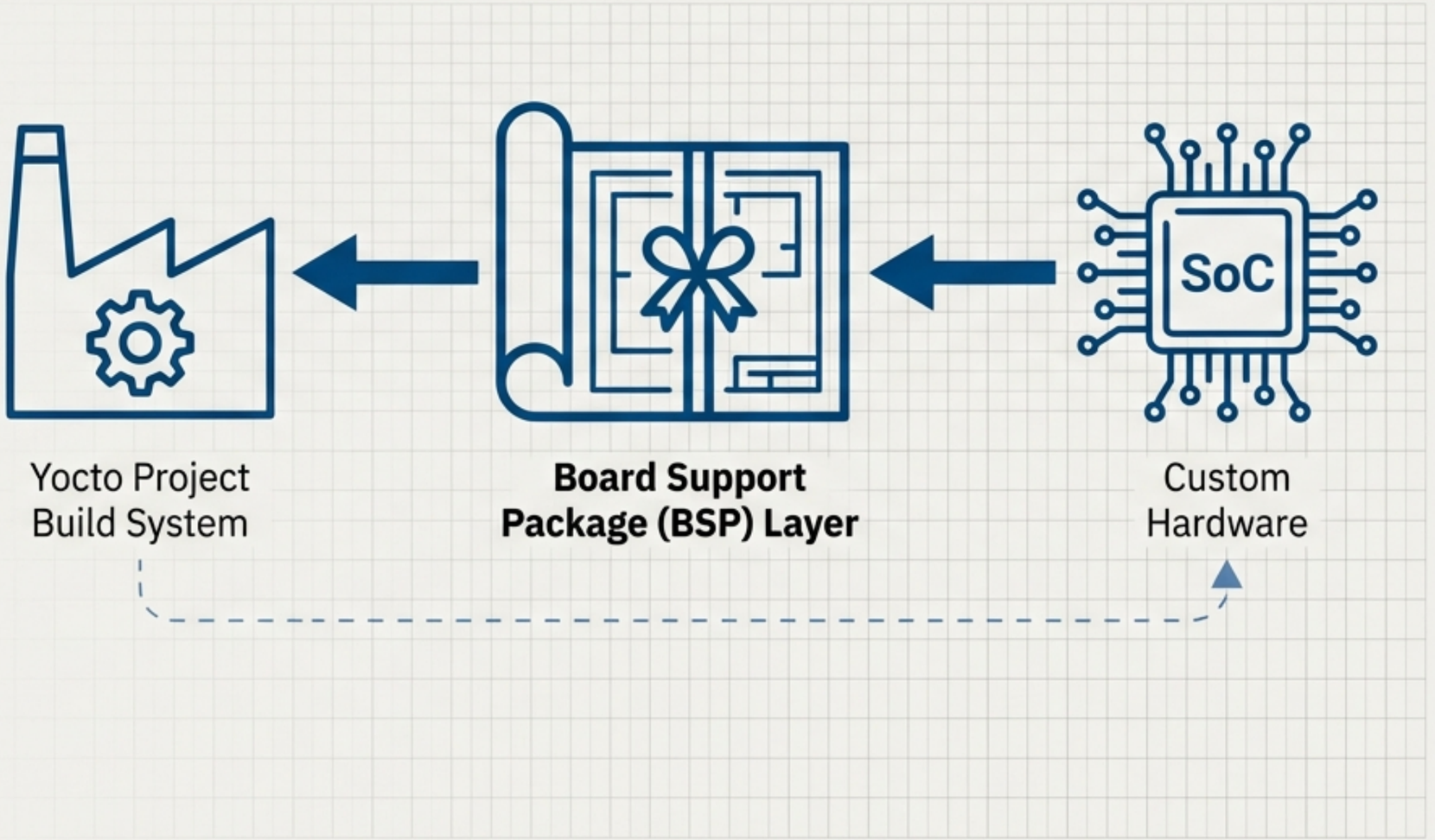
Transforming hardware-specific chaos into a standardised, buildable specification.

Every Custom Board Needs a Custom Linux. The BSP is the Plan.

A Board Support Package (BSP) is the collection of information that defines how to support a particular hardware device or platform.

It contains hardware features, kernel configurations, required drivers, and any additional software components needed beyond a generic Linux stack.

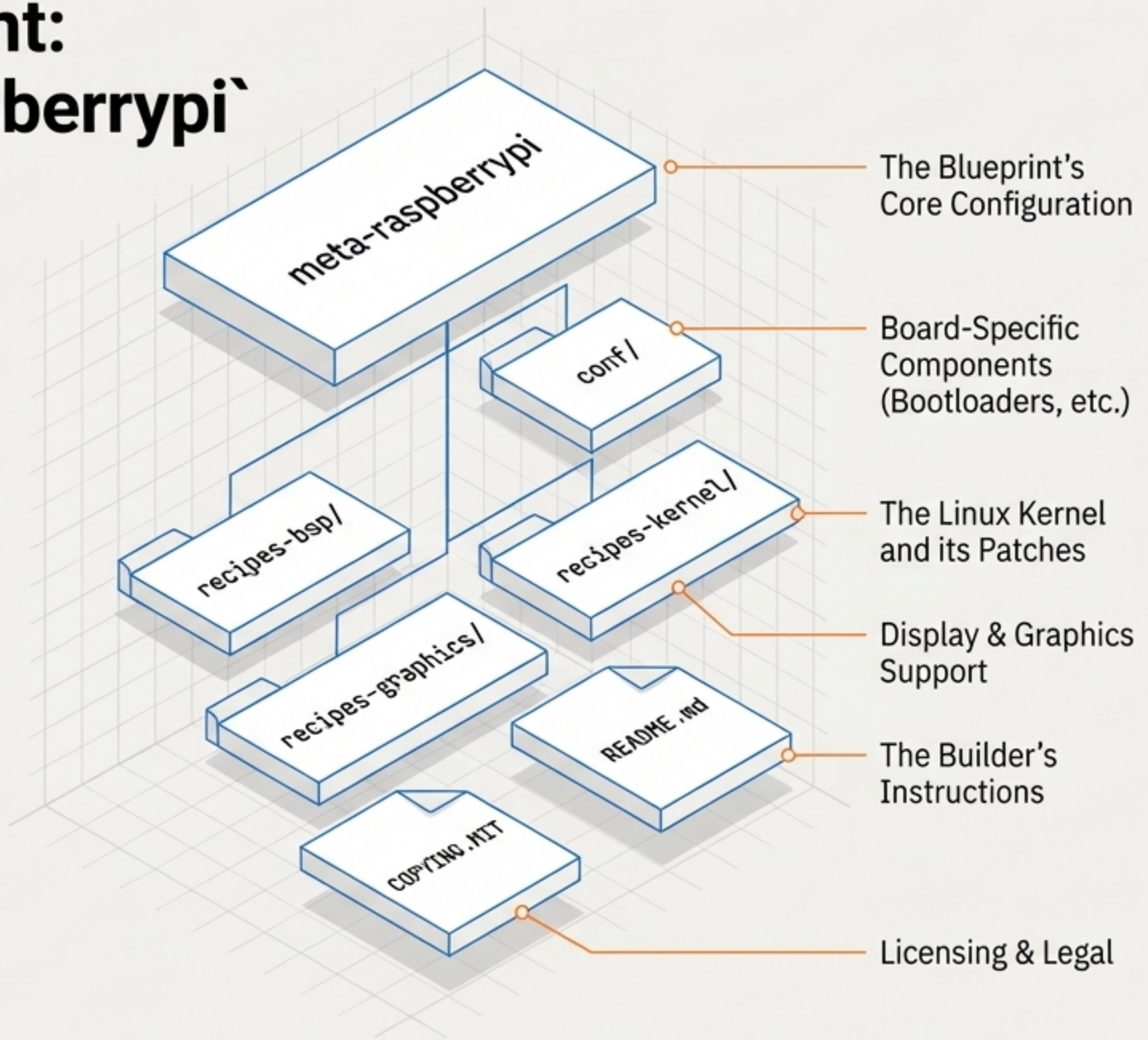
In the Yocto Project, the BSP is a specialised "layer" that acts as an architectural blueprint, instructing the build system how to construct the perfect OS for your hardware.



Analysing a Master Blueprint: The Anatomy of `meta-raspberrypi`

The best way to understand how a BSP is structured is to deconstruct a well-known, functional example. The `meta-raspberrypi` layer is a comprehensive BSP that supports the entire Raspberry Pi family. We will use it as our reference.

```
# Clone the reference BSP layer
$ git clone git://git.yoctoproject.org/meta-raspberrypi
```



The Layer's Identity: `conf/layer.conf`

This file is the first thing the build system looks for. It identifies the directory as a layer, defines its collection name, and sets its priority relative to other layers. It makes BitBake aware of all the recipes and configurations contained within.

```
# We have a conf and classes directory, append to BBPATH
BBPATH .= ":${LAYERDIR}"

# We have recipes, add them to the build files
BBFILES += "${LAYERDIR}/recipes*/*/*.bb \
            ${LAYERDIR}/recipes*/*/*.bbappend"

BBFILE_COLLECTIONS += "raspberrypi"
BBFILE_PATTERN_raspberrypi := "^${LAYERDIR}/"
BBFILE_PRIORITY_raspberrypi = "9"
```

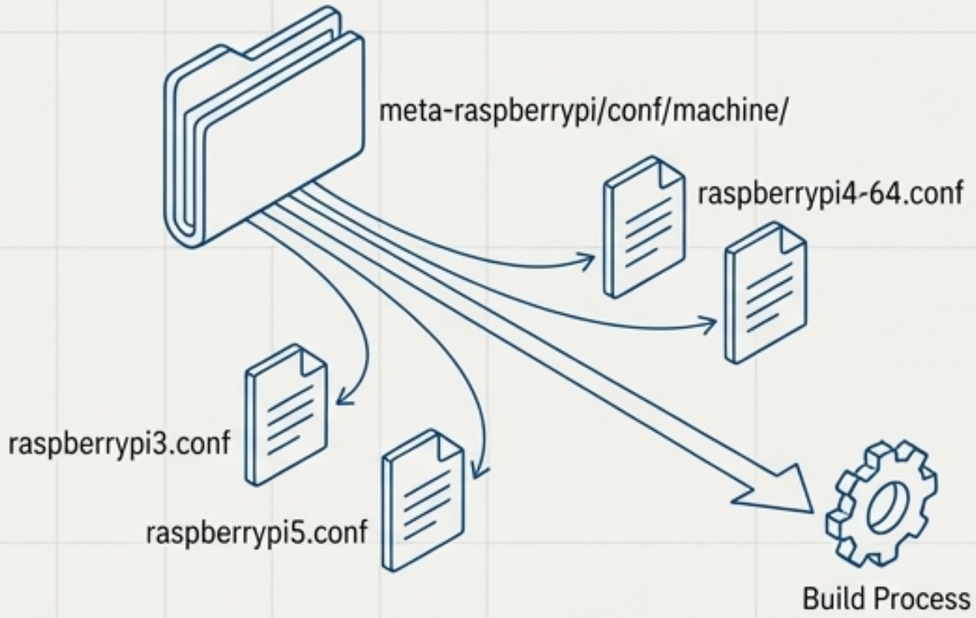
Adds layer's directories to the search path

A unique name for this layer's collection

Priority matters when recipes conflict

The Machine's DNA: `conf/machine/*.conf`

This is the heart of the BSP. Each `.conf` file defines a specific hardware target (a `MACHINE`). It specifies the kernel to use, required drivers, bootloader information, and processor-specific optimisations (tuning). A single BSP layer can support multiple machines.



DEFAULTTUNE

Defines package architecture and optimisation flags (e.g., `cortexa8hf-neon`).

PREFERRED_PROVIDER_virtual/kernel

Selects the kernel recipe to use (e.g., `"linux-yocto"`).

KERNEL_DEVICETREE

Specifies the device tree files (`.dtb`) required for the hardware.

UBOOT_MACHINE

Defines the configuration for the U-Boot bootloader.

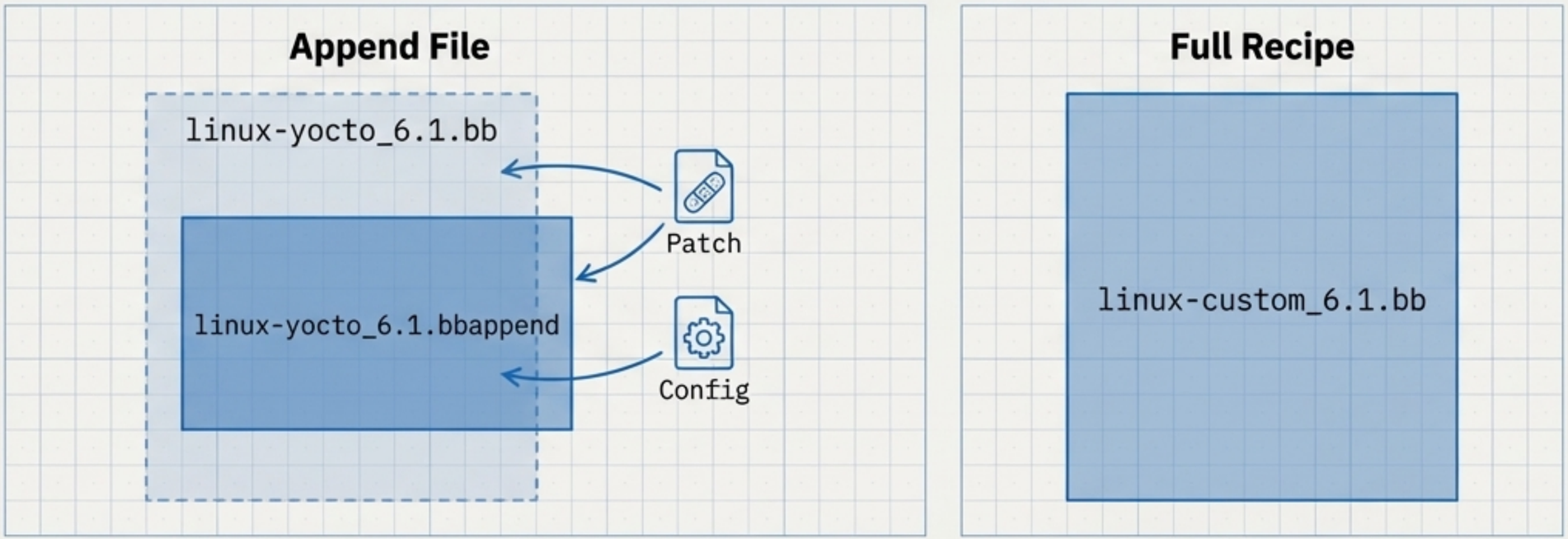
MACHINE_FEATURES

Lists hardware features to enable (e.g., `usbhost`, `vfat`, `alsa`).

The Kernel Connection: Customising `virtual/kernel`

A BSP must provide a Linux kernel for the board. This is typically done in one of two ways:

1. **Full Kernel Recipe (`.bb`):** For highly customised or non-mainline kernels, the BSP provides a complete recipe (e.g., `linux-raspberrypi_4.14.bb`).
2. **Append File (`.bbappend`):** More commonly, a BSP *appends* to a standard Yocto kernel recipe (like `linux-yocto`). This file adds machine-specific configurations, patches, and source code revisions.



Key Insight:
Using `.bbappend files is the recommended approach. It maximises reuse of the tested `linux-yocto` kernel and isolates your board-specific changes, simplifying maintenance.

A Professional Blueprint Requires Clear Documentation

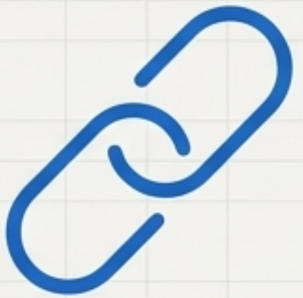
A compliant Yocto Project BSP is more than just configuration files. It must include clear documentation and licensing to be usable by others. These files are essential for collaboration and distribution.

`README` File

- Must contain: Hardware description, build dependencies (other layers), build/boot instructions, and maintainer contact info.

License File

- Must contain: A license for the BSP *metadata itself* (e.g., `COPYING.MIT`). This is separate from the licenses of the software being built.



Dependencies

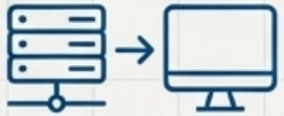
- Must be explicitly listed in the `README` and often codified in `conf/layer.conf` using `LAYERDEPENDS`. This ensures a reproducible build environment.

Drawing Your Own Blueprint: The BSP Creation Workflow

Creating a new, compliant BSP layer follows a structured process. The Yocto Project provides tools to automate the initial scaffolding, letting you focus on the hardware-specific details.

1

Prepare the Host

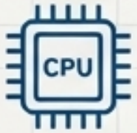


Set up your build environment and clone the `poky` repository.

2

Create the Layer

Use the `bitbake-layers create-layer` script to generate the standard directory structure.
`bitbake-layers create-layer meta-myboard`



3

Define the Machine

Create a new `conf/machine/your-machine.conf` file.

4

Configure the Kernel

Create a kernel recipe or a `.bbappend` file to tailor the Linux kernel.
`linux-yocto_%.bbappend`

5

Add Components

Add recipes for bootloaders, drivers, or other required packages.

6

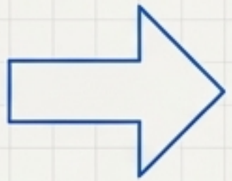
Document

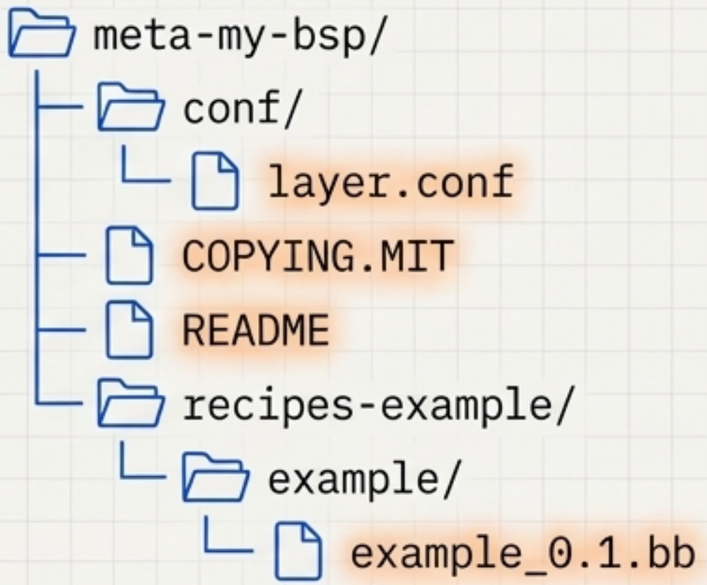
Create the `README` and License files.
`README` , `COPYING.MIT`

Step 1: Create the Layer Scaffolding

First, ensure your build environment is initialised. Then, use the `bitbake-layers create-layer` command. This tool creates a new directory with a correctly formatted `conf/layer.conf` and example recipes.

```
# 1. Initialise the build environment from
your poky directory
$ source oe-init-build-env

# 2. Create the new layer
$ bitbake-layers create-layer meta-my-bsp
```





You will then add this new layer to your `bblayers.conf` file to make the build system aware of it.

Step 2: Define Your Machine in `conf/machine/`

Create a file named `meta-my-bsp/conf/machine/my-machine-name.conf`. This file is where you will define all the hardware-specific attributes for your board.

```
# --- Core Board Definition ---
DESCRIPTION = "Configuration for My Custom Board"
DEFAULTTUNE ?= "cortexa8hf-neon"
include conf/machine/include/arm/armv7a/tune-cortexa8.inc

# --- Kernel Selection ---
PREFERRED_PROVIDER_virtual/kernel ?= "linux-yocto"
PREFERRED_VERSION_linux-yocto ?= "6.1%"
KERNEL_IMAGETYPE = "zImage"
KERNEL_DEVICETREE = "my-custom-board.dtb"

# --- Bootloader Configuration ---
PREFERRED_PROVIDER_virtual/bootloader ?= "u-boot"
SPL_BINARY = "MLO"
UBOOT_MACHINE = "am335x_evm_defconfig"

# --- Image & Filesystem ---
IMAGE_FSTYPES += "wic"
WKS_FILE ?= "my-board.wks"
IMAGE_BOOT_FILES = "u-boot.img MLO ${KERNEL_IMAGETYPE} ${KERNEL_DEVICETREE}"

# --- Hardware Features ---
MACHINE_FEATURES = "usbgadget usbhost vfat alsa"
SERIAL_CONSOLES = "115200;ttyO0"
```

Step 3: Tailor the Kernel with a `.bbappend`

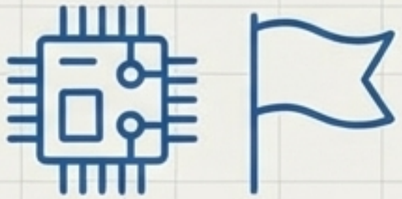
Create a file in your layer that appends to your chosen **kernel recipe**. The filename must match exactly. For `linux-yocto_6.1.bb`, create `recipes-kernel/linux/linux-yocto_6.1.bbappend`. This file specifies the kernel source branch, commit ID, and machine compatibility.

```
# In meta-my-bsp/recipes-kernel/linux/linux-yocto_6.1.bbappend

FILESEXTRAPATHS:prepend := "${THISDIR}/files:"

KBRANCH:my-machine-name = "v6.1/standard/base"

KMACHINE:my-machine-name = "my-machine"

SRCREV_machine:my-machine-name = "423e1996694b61fbfc8ec3bf062fc6461d64fde1"

COMPATIBLE_MACHINE:my-machine-name = "my-machine-name"
```

- Path for patches
- Branch in the `linux-yocto` git repo
- Machine name as known by the kernel sources
- The exact git commit hash for the kernel source
- Ensures this append only applies to your machine

A Note on Commercial Licenses

Some BSPs require components with specific End User License Agreements (EULAs). The Yocto Project manages this through a whitelisting mechanism. The build will stop and inform you of any required licenses.



1. Recipe sets flag

Recipes for encumbered software set a `LICENSE_FLAGS` variable (e.g., `LICENSE_FLAGS = "commercial_my-driver\"`).

2. User accepts flag

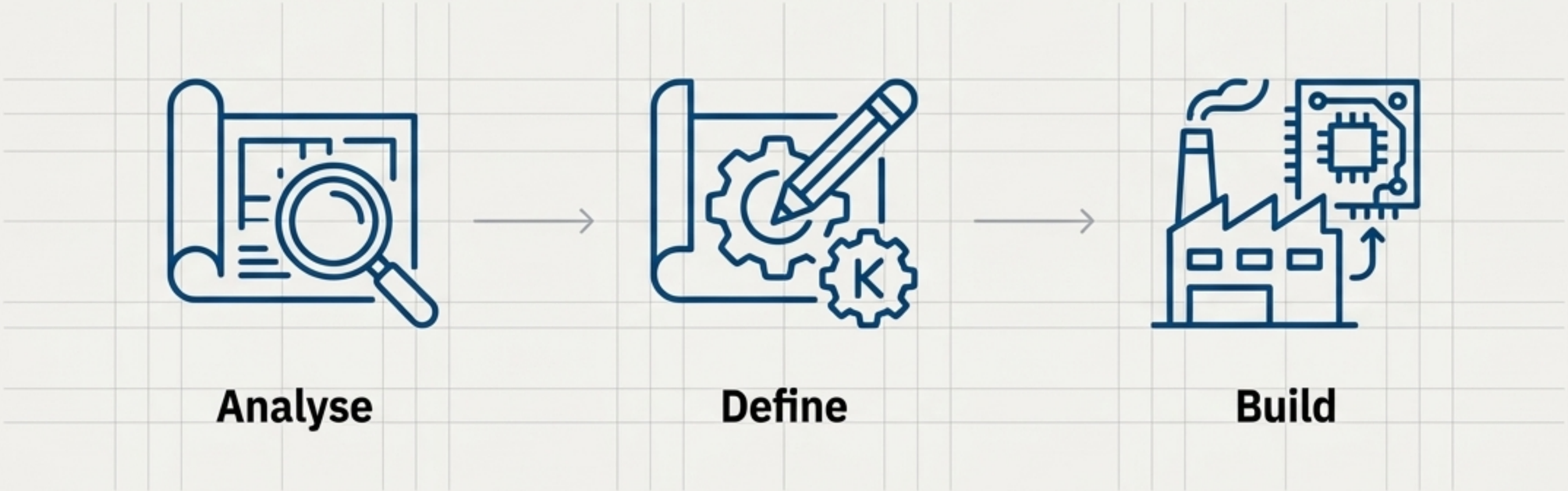
To proceed with the build, you must explicitly accept the license by adding the flag to your `local.conf` file.

```
# In your build/conf/local.conf file
# This signifies you have read and agree to the license terms.
LICENSE_FLAGS_ACCEPTED += "commercial_my-driver"
```

This system prevents accidental inclusion of encumbered IP and ensures license compliance.

From Blueprint to Build: The Power of a Well-Defined BSP

A Board Support Package transforms hardware enablement from an ad-hoc process into a disciplined, architectural practice. By deconstructing a master blueprint like `meta-raspberrypi`, you learn the components needed to draw your own.



Your BSP is the definitive, reproducible, and maintainable plan for your product's software foundation. Build it with the precision it deserves.